

EVENTSMK

1. Инсталација на Python.
2. Поставување како environment variable во полето User Variables.
3. Клонирање на репозиториумот од GitHub.

```
PS D:\Workspace> git clone https://github.com/iSqloud7/EventsAggregator-Project.git
Cloning into 'EventsAggregator-Project' ...
remote: Enumerating objects: 736, done.
remote: Counting objects: 100% (51/51), done.
remote: Compressing objects: 100% (41/41), done.
remote: Total 736 (delta 21), reused 15 (delta 10), pack-reused 685 (from 2)
Receiving objects: 100% (736/736), 13.14 MiB | 4.28 MiB/s, done.
Resolving deltas: 100% (138/138), done.
PS D:\Workspace> cd .\EventsAggregator-Project\
PS D:\Workspace\EventsAggregator-Project> git branch
* master
PS D:\Workspace\EventsAggregator-Project>
```

4. Во IntelliJ: креирање и активирање на venv.

- PS D:\Workspace\EventsAggregator-Project> python -m venv venv
- PS D:\Workspace\EventsAggregator-Project> .\venv\Scripts\activate

```
PS D:\Workspace\EventsAggregator-Project> python -m venv venv
PS D:\Workspace\EventsAggregator-Project> .\venv\Scripts\activate
(venv) PS D:\Workspace\EventsAggregator-Project>
```

5. Инсталација на requirements.txt.

- (venv) PS D:\Workspace\EventsAggregator-Project> pip install -r .\requirements.txt

```
(venv) PS D:\Workspace\EventsAggregator-Project> pip install -r .\requirements.txt
```

6. Конфигурација:

- EventsAggregator-Project\.env:

```
DB_URL=postgresql+psycopg2://postgres:postgres@localhost:5432/events-aggregator-db
```

- EventsAggregator-Project\backend\src\main\resources\application.properties:

```
# DATABASE CONFIG
spring.application.name=backend
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost:5432/events-aggregator-db
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA / HIBERNATE
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

# OPTIONAL
spring.jpa.open-in-view=false

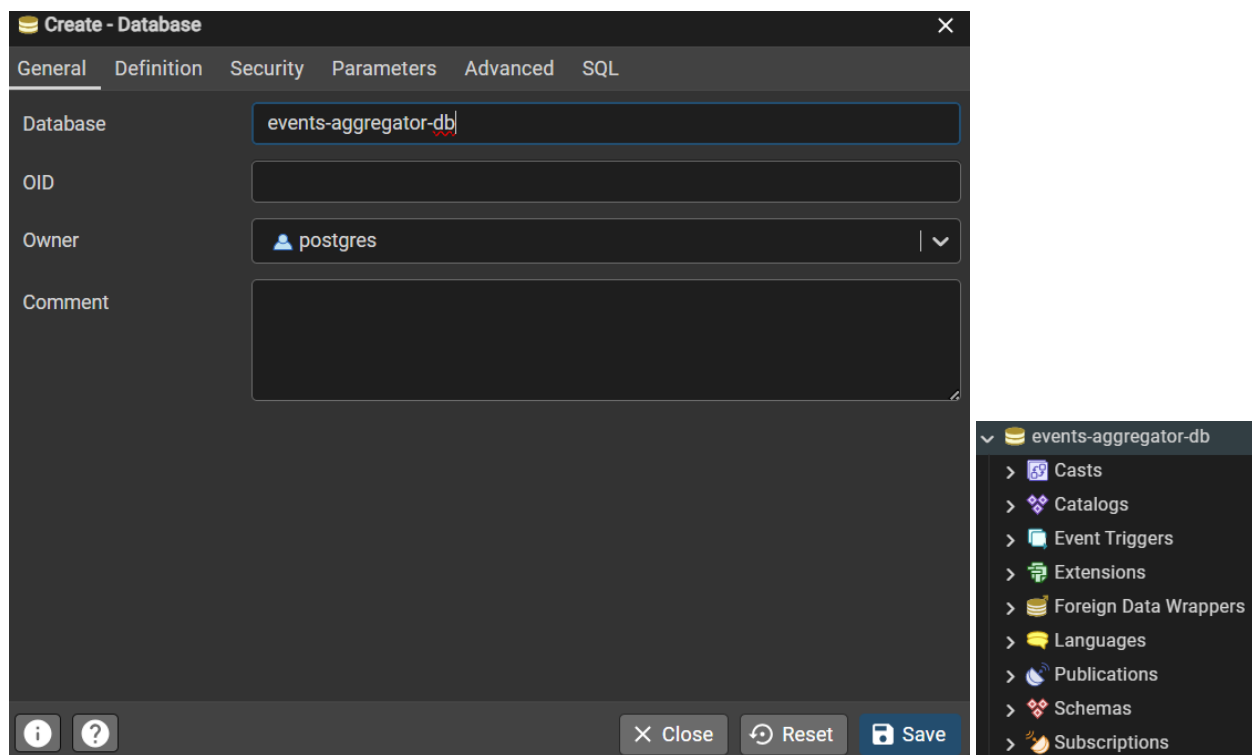
# logging.level.org.springframework=DEBUG
```

7. Unlink Maven Projects на pom.xml во EventsAggregator-Project\backend\pom.xml.

8. Link Maven Projects на pom.xml во EventsAggregator-Project\backend\pom.xml.

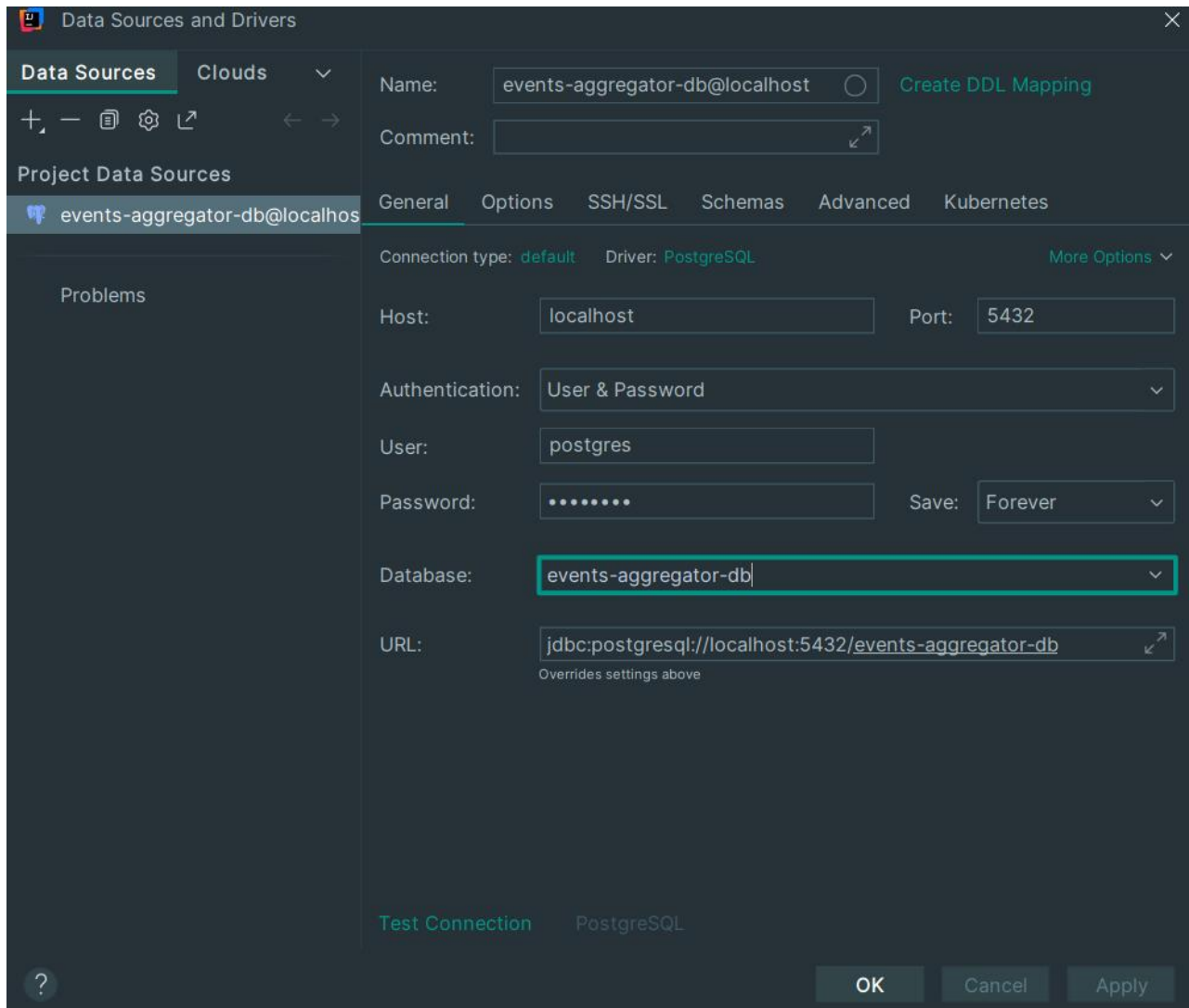
9. Креирање на базата во pgAdmin.

Database->Create->Database



10. Креирање на базата во IntelliJ.

DataseIcon->+->DataSource->PostgreSQL



TestConnection->Apply->OK

10. Креирање на events табелата во базата и популаризирање со events.

```
(venv) PS D:\Workspace\EventsAggregator-Project> python .\scraper_postgres_connector\main.py
```

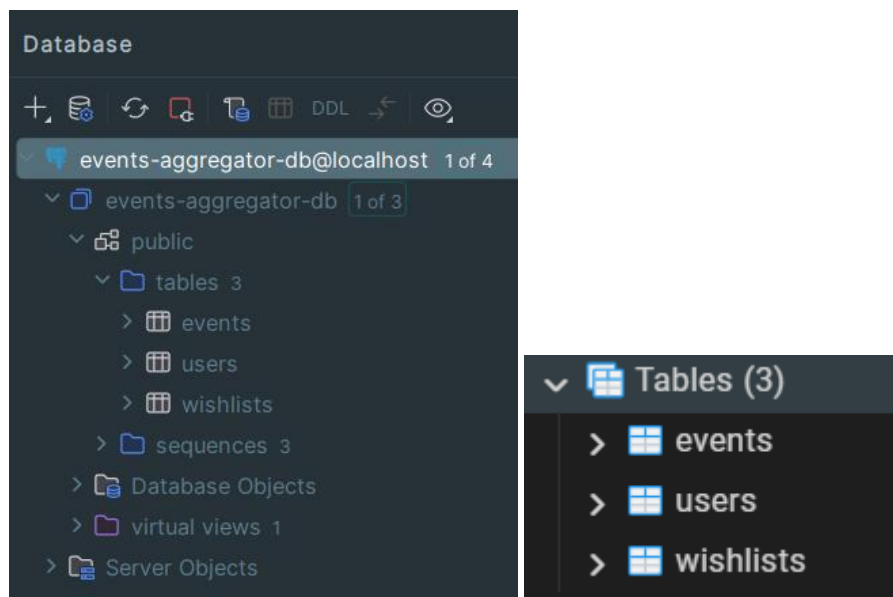
```
(venv) PS D:\Workspace\EventsAggregator-Project> python .\scraper_postgres_connector\main.py
```

11. Ажурирање на податоците во базата.

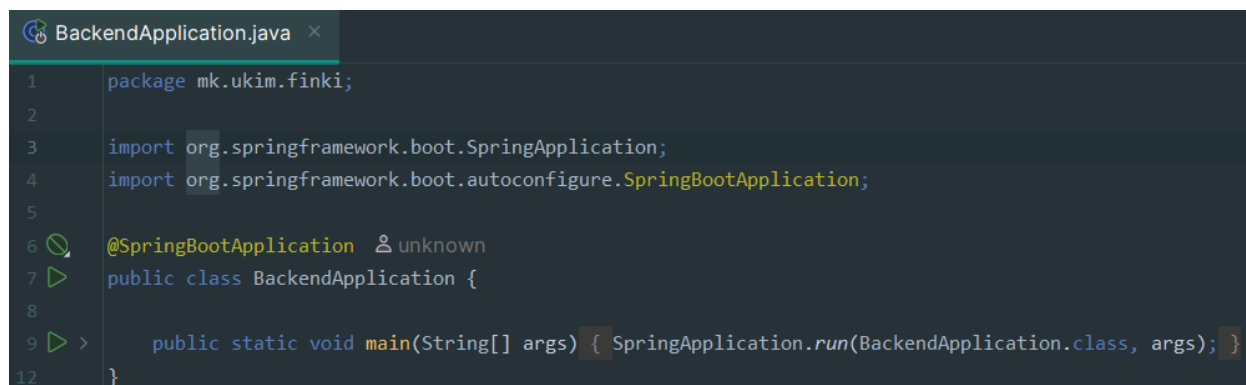
```
(venv) PS D:\Workspace\EventsAggregator-Project> python .\mktickets_scraper\scheduler\scheduler.py
```

```
(venv) PS D:\Workspace\EventsAggregator-Project> python .\mktickets_scraper\scheduler\scheduler.py
```

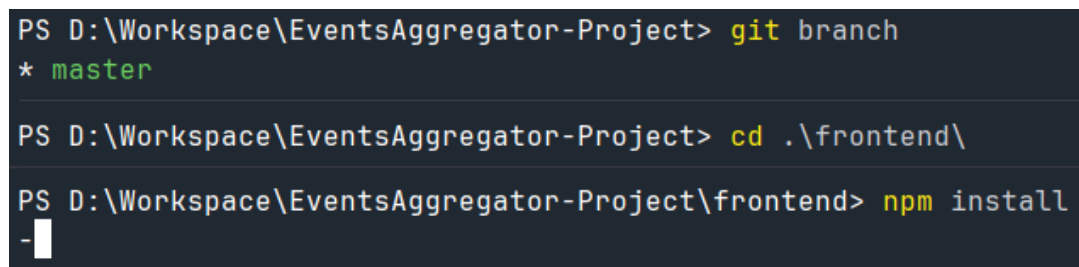
12. После извршените команди, треба да изгледа вака:



13. Стартување на back-end апликацијата: BackendApplication.

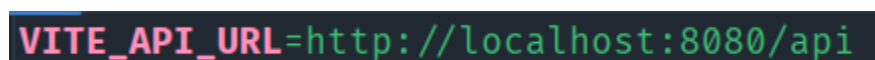


14. Потребни подготовки на front-end апликацијата.



15. Конфигурација:

- EventsAggregator-Project\frontend\.env:



16. Стартување на front-end апликацијата.

PS D:\Workspace\EventsAggregator-Project\frontend> npm run dev

```
VITE v5.4.21 ready in 599 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

17. Успешно стартување на апликацијата.

